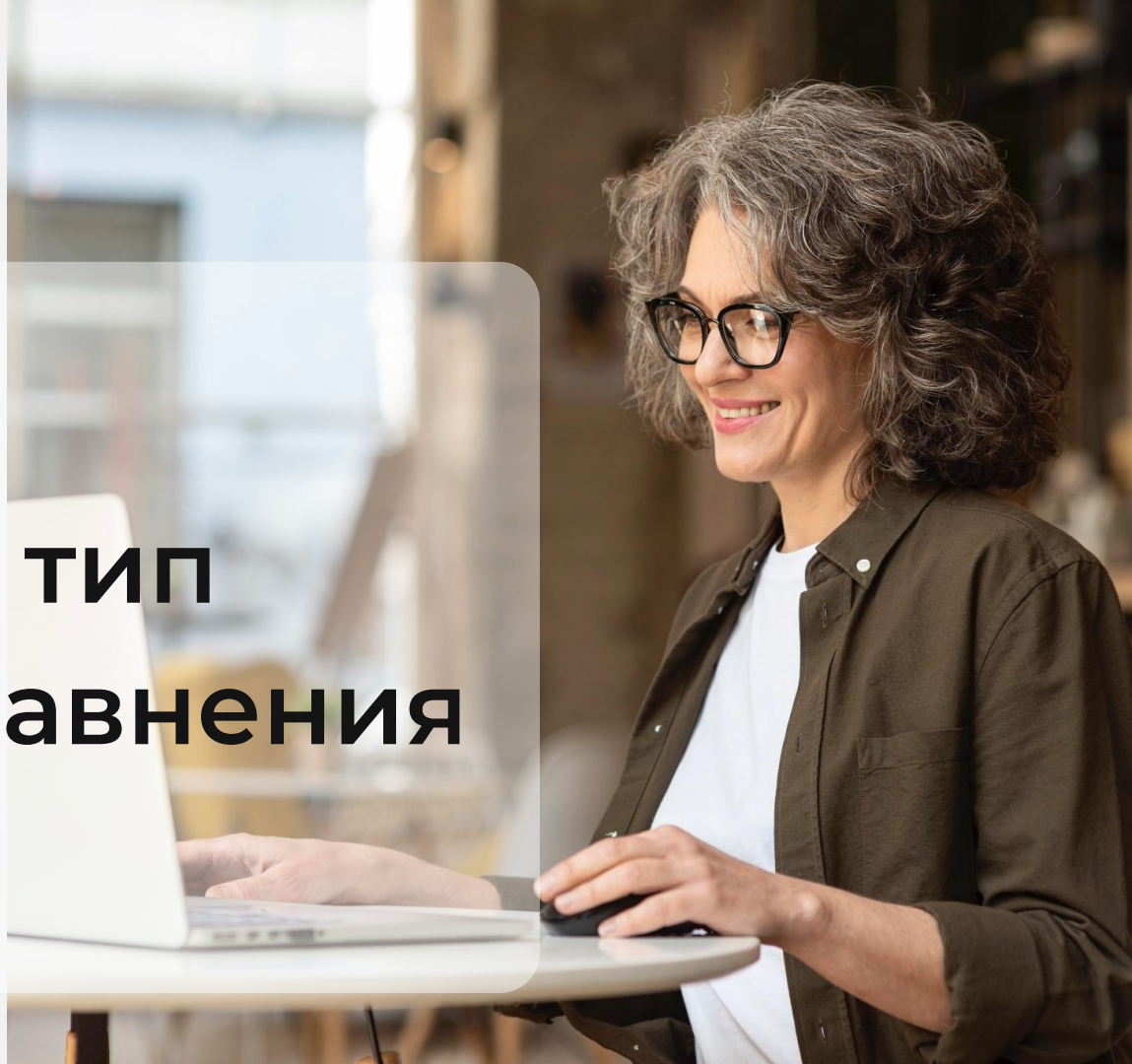


Python

Логический тип данных и сравнения



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

- Числа
- Арифметические операции с числами
- ZeroDivisionError
- Приоритет математических операций и скобок
- Неизменяемые типы данных
- Операторы приращения
- Множественное присваивание
- Преобразование типов, ValueError

План занятия

- Логический тип данных
- Приведение типов к bool
- Операторы сравнения
- Логические операторы
- Комбинирование логических операторов
- Приоритет логических операторов
- Таблица истинности
- Альтернативная интерпретация логических операторов
- Двойные неравенства



ОСНОВНОЙ БЛОК





**Логический тип
данных**



Логический тип данных

Используется для представления двух возможных значений: True (истина) и False (ложь).



True и False

Эти значения помогают программе принимать решения. Например, в зависимости от того, истинное значение или ложное, программа может выполнить разные действия.

Полезно знать



Логические значения

Они могут быть созданы напрямую или быть результатом логических операций, сравнений или приведения типов к **bool**.

Создание напрямую:

`isFinished = True`

`hasAccess = False`



ВОПРОСЫ





Приведение типов к bool

Значения



True

Любое ненулевое число, непустая строка, список, кортеж и т.д.

False

None, 0, пустые последовательности (например, [], {}, "").

Преобразования в bool



Автоматическое

Python автоматически преобразует значения в True или False, когда это необходимо.

Явное (ручное)

Можно использовать функцию bool.

Автоматическое преобразование



Условия if



Циклы



Логические операции

Явное (ручное) преобразование



```
print(bool(5))           # True (ненулевое число)
print(bool(0))           # False
print(bool(None))        # False
print(bool([]))          # False (пустой список)
print(bool("Hello"))     # True (непустая строка)
```




ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Определите, какие из этих значений преобразуются в True при использовании функции `bool()`.

- a. 42
- b. 0
- c. -2
- d. ""
- e. "0"
- f. "Python"
- g. "False"
- h. []
- i. None
- j. 3.14



Выберите правильный вариант ответа

Определите, какие из этих значений преобразуются в True при использовании функции `bool()`.

a. 42

b. 0

c. -2

d. ""

e. "0"

f. "Python"

g. "False"

h. []

i. None

j. 3.14



ВОПРОСЫ





Операторы сравнения





Операторы сравнения

Эти операторы применяются для проверки равенства, неравенства, а также для сравнения величин (больше, меньше и т.д.).

Оператор	Описание	Пример	Результат
==	Равно	5 == 5	True
!=	Не равно	5 != 5	False
>	Больше	3 > 5	False

Оператор	Описание	Пример	Результат
<	Меньше	$4 < 6$	True
>=	Больше или равно	$5 >= 5$	True
<=	Меньше или равно	$3 <= 2$	False

Операторы сравнения с числами



Оператор == (равно)

Сравнивает два числа на равенство

Пример

```
a = 10  
b = 10  
print(a == b)  # True, так как a равно b
```

Операторы сравнения с числами



Оператор != (не равно):

Проверяет, не равны ли два числа.

Пример

```
a = 10
b = 5
print(a != b)  # True, так как a не
               равно b
```

Операторы сравнения с числами



Оператор > (больше)

Проверяет, больше ли одно число другого.

Пример

```
a = 8
b = 10
print(a > b) # False, так как a меньше b
```

Операторы сравнения с числами



Оператор < (меньше)

Проверяет, меньше ли одно число другого.

Пример

```
a = 5  
b = 7  
print(a < b) # True, так как a меньше  
b
```

Операторы сравнения с числами



Оператор `>=` (больше или равно)

Проверяет, больше или равно ли одно число другому.

Пример

```
a = 5
b = 5
print(a >= b)  # True, так как a равно b
```

Операторы сравнения с числами



Оператор `<=` (меньше или равно)

Проверяет, меньше или равно ли одно число другому.

Пример

```
a = 3
b = 4
print(a <= b)  # True, так как a меньше b
```



ВОПРОСЫ





ЗАДАНИЕ



Соотнесите оператор сравнения с его результатом

1. $5 \neq 3$

a. False

2. $7 < 6$

b. True

3. $8 \geq 8$

4. $4 > 2$

Соотнесите оператор сравнения с его результатом

1. $5 \neq 3$

b. True

2. $7 < 6$

a. False

3. $8 \geq 8$

b. True

4. $4 > 2$

a. True



ВОПРОСЫ





Логические операторы



Логические операторы

Используются для выполнения логических операций над значениями или выражениями.

Оператор	Описание	Пример	Результат
and	Логическое "И": возвращает True, если оба условия истинны	True and False	False
or	Логическое "ИЛИ": возвращает True, если хотя бы одно условие истинно	True or False	True
not	Логическое "НЕ": возвращает противоположное значение	not True	False

Логические операторы



Оператор and (логическое "И")

Оператор and возвращает True, если оба условия истинны.

Если хотя бы одно из условий ложно, результатом будет False.

Пример

```
a = 5
b = 10
```

Проверяем что оба числа положительные

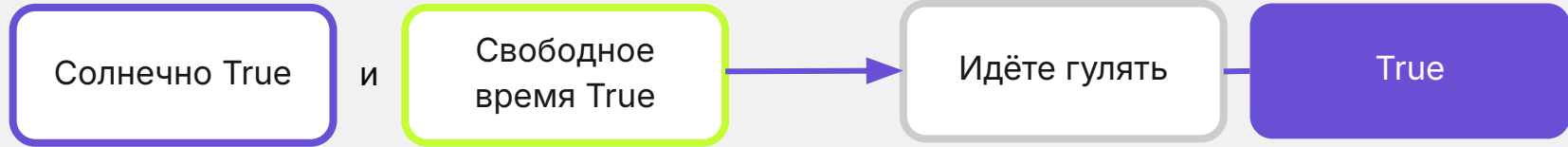
```
print(a > 0 and b > 0) # True, так как оба условия истинны
```


Пример из жизни для логического оператора **and**

Представьте, что вы хотите пойти на прогулку, но только если **солнечно** и у вас есть **свободное время**.

Оба условия должны быть истинными, чтобы вы пошли гулять.





Это как сказать:

"Я пойду гулять, если одновременно и погода хорошая, и у меня есть свободное время."

Логические операторы



Оператор or (логическое "ИЛИ")

Оператор or возвращает True, если хотя бы одно из условий истинно.

Если оба условия ложны, результатом будет False.

Пример

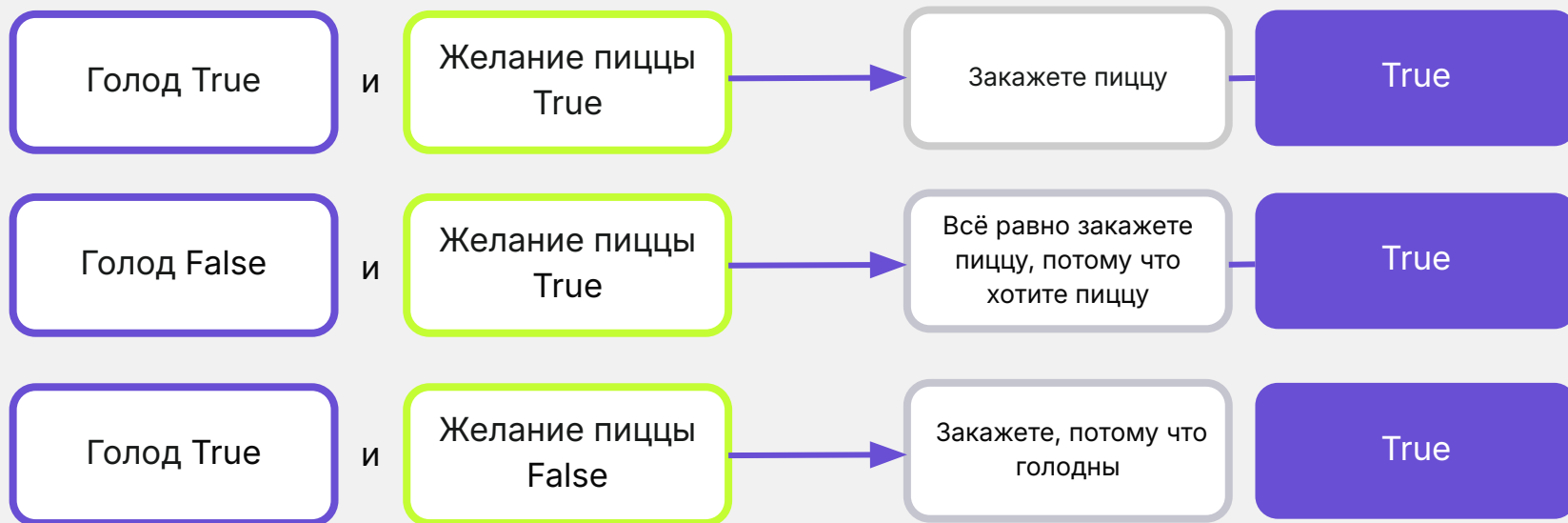
```
a = -5
b = 10
# Проверяем что хотя бы одно из чисел
# положительное
print(a > 0 or b > 0)  # True, так как b > 0 истинно
```

Пример из жизни для логического оператора or

Теперь представьте, что вы хотите заказать пиццу, но вы сделаете заказ, если у вас либо **голод**, либо **желание пиццы**.

В этом случае, достаточно, чтобы хотя бы одно из условий было истинным.





Это как сказать: "Я закажу пиццу, если либо я голоден, либо у меня есть настроение её съесть."

Логические операторы



Оператор not (логическое "НЕ")

Оператор not возвращает противоположное значение.

Если выражение истинно, not превращает его в ложное, и наоборот.

Пример

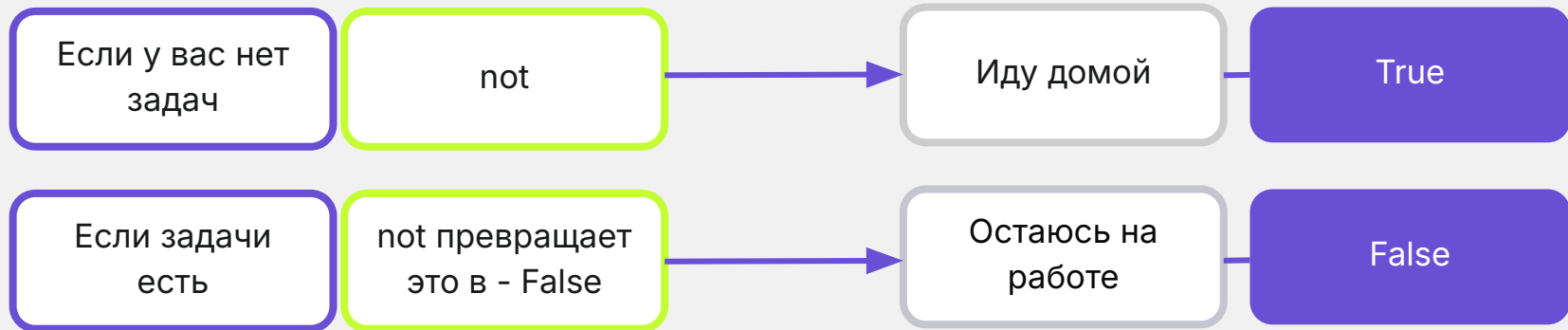
```
a = False
```

```
print(not a) # True, так как это обратное значение от False
```


Пример из жизни для логического оператора `not`

Представьте, что вы на работе и планируете уйти домой. Вы пойдёте домой только в том случае, если у вас нет задач.

Здесь оператор `not` работает как отрицание: если задач нет, то вы идёте домой.





Это как сказать: "Я иду домой, если у меня нет задач", что является логическим отрицанием выполнения работы.



ВОПРОСЫ





ЗАДАНИЕ



Соотнесите оператор сравнения с его результатом

- | | |
|-------------------|----------|
| 1. True and False | a. True |
| 2. True or False | b. False |
| 3. not True | |

Соотнесите оператор сравнения с его результатом

- | | |
|-------------------|----------|
| 1. True and False | b. False |
| 2. True or False | a. True |
| 3. not True | b. False |



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
x = 5
```

```
print(not x > 3)
```

- a. True
- b. False
- c. Ошибка
- d. None



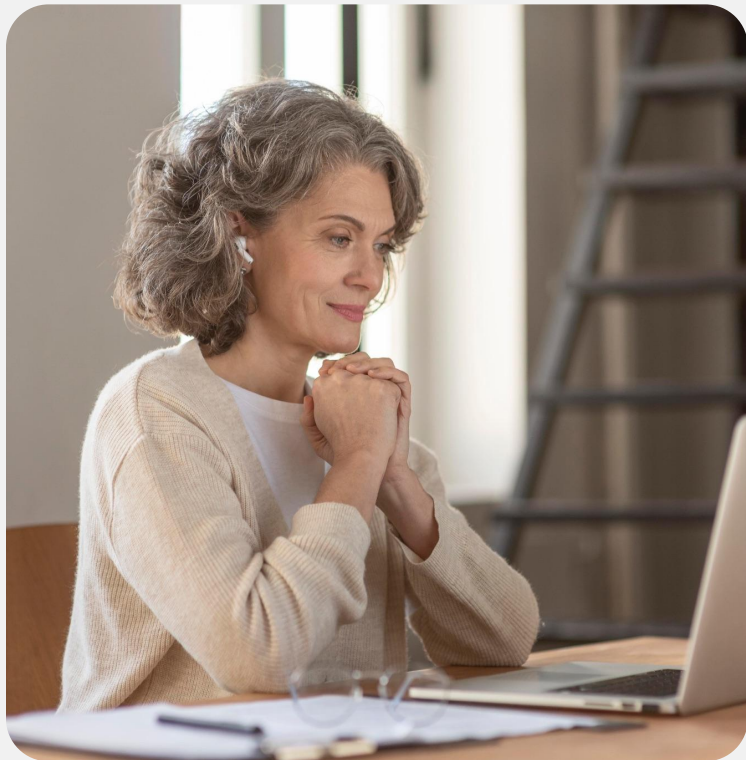
Выберите правильный вариант ответа

Какой результат выведет следующий код?

`x = 5`

`print(not x > 3)`

- a. True
- b. False**
- c. Ошибка
- d. None



Выберите правильный вариант ответа

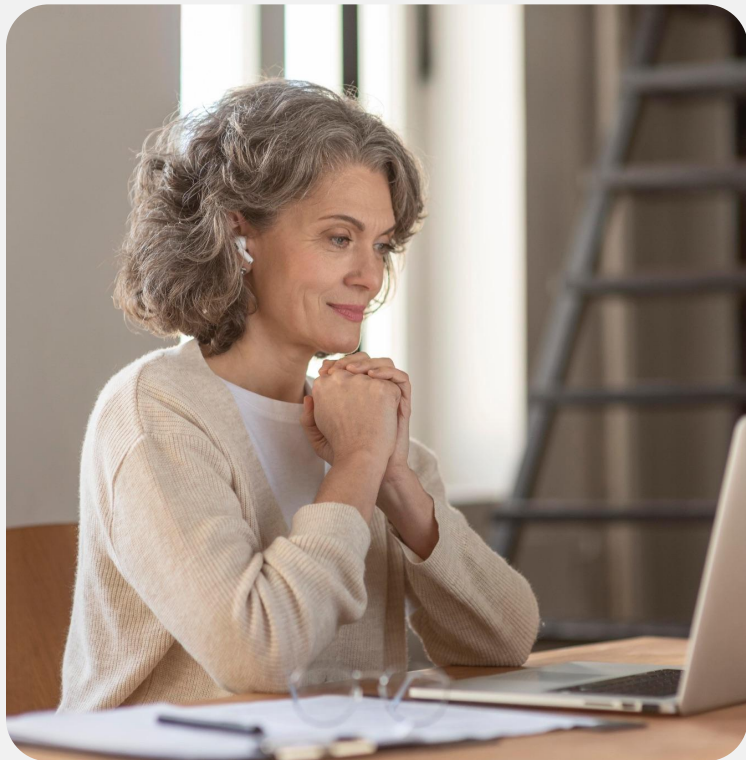
Какой результат выведет следующий код?

a = 5

b = 10

print(a < b and b < 10)

- a. True
- b. False
- c. Ошибка
- d. None



Выберите правильный вариант ответа

Какой результат выведет следующий код?

a = 5

b = 10

print(a < b and b < 10)

- a. True
- b. False**
- c. Ошибка
- d. None



ВОПРОСЫ





Комбинирование
логических
операторов

Комбинирование



Возможности

Вы можете комбинировать несколько логических операторов в одном выражении для создания более сложных условий.

В этом примере результат будет True, потому что оба числа больше 0 (условие с and истинно), и также `a == 5` (условие с or истинно).

Пример

```
a = 5
```

```
b = 10
```

```
print(a > 0 and b > 0 or a == 5)
```



ВОПРОСЫ





Приоритет логических операторов



Приоритет операторов

В Python логические операторы имеют определённый порядок выполнения (приоритет).

Порядок приоритета



1. not (логическое "НЕ") — выполняется первым.
2. and (логическое "И") — выполняется вторым.
3. or (логическое "ИЛИ") — выполняется последним.

Порядок приоритета

Пример:

a = True

b = True

c = True

print(not a or b and c)

В этом выражении сначала выполнится оператор not, затем and, и последним — or:

1. not a or b and c # Подставим значения переменных
2. not True or True and True # not True → False
3. False or True and True # True and True → True
4. False or True # False or True → True
5. True



Скобки для приоритета

Если вы хотите изменить порядок выполнения логических операторов, можно использовать скобки, чтобы явно указать, какие операции должны быть выполнены в первую очередь, так как скобки имеют наивысший приоритет.

Скобки для изменения приоритета

Пример:

result = not (a or b) and c

Теперь оператор or выполняется первым (из-за скобок), затем not, и в последнюю очередь — and:

1. not (a or b) and c # Подставим значения переменных
2. not (**False or True**) and True # False or True → True
3. **not True** and True # not True → False
4. **False and True** # False and True → False
5. **False**

Математические операции и операторы сравнения



Приоритет

Математические операции имеют более высокий приоритет, чем операторы сравнения. Это значит, что сначала Python выполнит все вычисления, а затем сравнит результат.

Пример

```
result = 3 + 2 > 4 # Сначала выполняется  
сложение (3 + 2), затем результат сравнивается  
с 4
```

```
print(result)      # True, так как 5 > 4
```

Пример из жизни для нескольких условий

Использование and и or

Предположим, вы хотите узнать, можно ли пойти гулять.

Условия: (погода хорошая **или** день выходной), **и** у вас есть свободное время.

```
good_weather = True
```

```
is_weekend = False
```

```
free_time = True
```

```
can_go_out = (good_weather or is_weekend) and  
free_time
```

```
print(can_go_out) # Результат: True
```

Пример из жизни для нескольких условий

Использование `not` с другими операторами

Допустим, вы планируете поехать в отпуск, если у вас **нет** работы **и** достаточно денег.

```
have_work = True
```

```
enough_money = True
```

```
can_go_vacation = not have_work and enough_money
```

```
print(can_go_vacation) # Результат: False
```


Пример из жизни для нескольких условий

Использование not с другими операторами

- not have_work → превращает True в False (у вас есть работа, значит, вы не можете поехать).
- and enough_money → проверяет, достаточно ли денег (результат: False, так как денег недостаточно).
- Окончательный результат: False, вы не можете поехать в отпуск.

```
have_work = True
```

```
enough_money = True
```

```
can_go_vacation = not have_work and enough_money
```

```
print(can_go_vacation) # Результат: False
```



ВОПРОСЫ





Таблица истинности



Таблица истинности

Показывает результат работы логических операторов для всех возможных комбинаций значений True и False.

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True



ВОПРОСЫ





**Альтернативная
интерпретация
логических операторов**



Интерпретация логических операторов

В Python логические операторы `and`, `or` и `not` можно сопоставить с арифметическими операциями.

A	B	A and B ($A * B$)	A or B ($A + B$)	not A (инверсия A)
True	True	True ($1 * 1$)	True ($1 + 1$)	False ($1 \rightarrow 0$)
True	False	False ($1 * 0$)	True ($1 + 0$)	False ($1 \rightarrow 0$)
False	True	False ($0 * 1$)	True ($0 + 1$)	True ($0 \rightarrow 1$)
False	False	False ($0 * 0$)	False ($0 + 0$)	True ($0 \rightarrow 1$)



ВОПРОСЫ





Двойные неравенства



Двойные неравенства

В Python можно использовать двойные неравенства, что позволяет проверять, попадает ли число в определённый диапазон значений.

Синтаксис двойных неравенств:



```
min_value < variable < max_value
```

Синтаксис двойных неравенств



Проверка двух условий

1. $1 < x$ — проверка, что x больше 1.
2. $x \leq 10$ — проверка, что x меньше или равно 10.

Пример

$x = 5$

```
print("1 < x <= 10") # x находится между 1 и 10  
(включительно)
```

Эквивалент без двойного неравенства



Объяснение

Использование двойных неравенств эквивалентно коду с логическим оператором `and`.

Пример

```
x = 5
```

```
print(1 < x and x <= 10) # x находится между 1 и 10 (включительно)
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Что произойдёт при выполнении следующего кода?

```
x = 3
print(1 < x < 5)
```

- a. True
- b. False
- c. Ошибка
- d. None

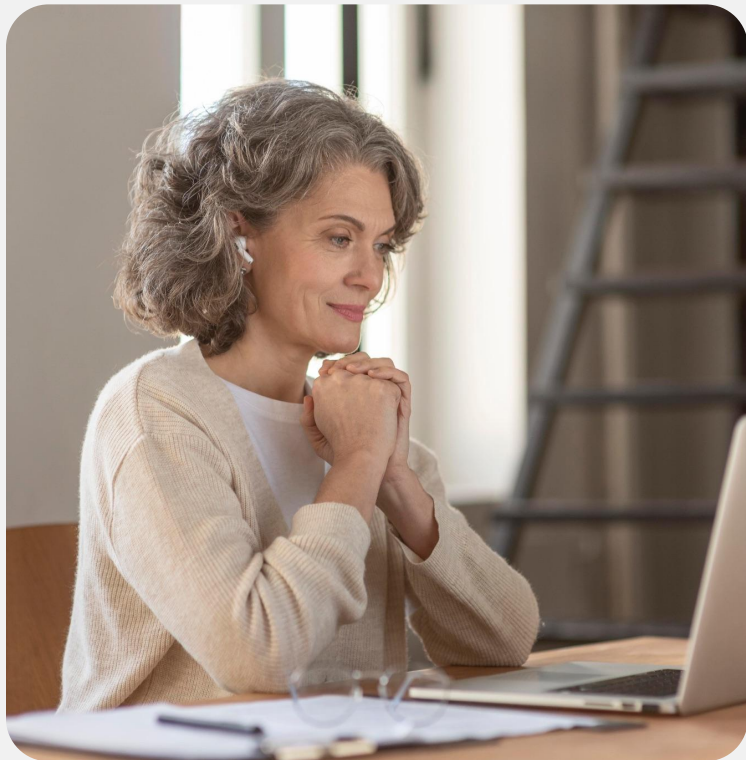


Выберите правильный вариант ответа

Что произойдёт при выполнении следующего кода?

```
x = 3
print(1 < x < 5)
```

- a. **True**
- b. False
- c. Ошибка
- d. None



Выберите правильный вариант ответа

Что произойдёт при выполнении следующего кода?

a = False

b = True

c = True

print(not a or b and c)

- a. True
- b. False
- c. Ошибка
- d. None



Выберите правильный вариант ответа

Что произойдёт при выполнении следующего кода?

a = False

b = True

c = True

print(not a or b and c)

a. True

b. False

c. Ошибка

d. None



ВОПРОСЫ



Домашнее задание

1. Проверка условий

Напишите программу, которая принимает на вход логические значения двух переменных (свет включён и окно открыто) и проверяет:

- Оба ли условия выполнены.
- Хотя бы одно из условий выполнено.

Домашнее задание

2. Стоимость мобильного тарифа

Напишите программу для расчёта стоимости использования мобильного тарифа:

- Базовая стоимость: 30 евро.
- Каждый мегабайт интернета сверх 500 МБ стоит 0.2 евро.

Программа должна запросить у пользователя количество использованных мегабайтов и вывести сколько всего он заплатил (базовый и переплата).

Заключение

